

Governing the KV Cache: Preventing Timing Side-Channel Leakage in Multi-Tenant LLM Inference

Tejasvi C. Addagada

Abstract

The key-value (KV) cache is the primary throughput optimization in modern large language model (LLM) inference, enabling prefix reuse across requests. In multi-tenant deployments, this cache is shared across tenants, creating a timing side channel: an adversarial tenant can reconstruct another tenant’s private prompt by probing cache-hit latency patterns. Three independent attacks exploit this channel—PROMPTPEEK (Wu et al., 2025), EarlyBird (Song et al., 2024), and InputSnatch (Zheng et al., 2024b)—and reach up to 100% attack success rate (ASR) against unprotected vLLM and SGLang deployments, with reported rates varying by cache architecture and prompt structure.

We present KVGov, the first unified governance layer that defends against all three attack families simultaneously. KVGov’s core mechanism is HMAC-keyed namespace isolation: a per-principal salt $\sigma_p = \text{HMAC}_K(\text{secret}, \text{principal.id})$ seeds the block-hash chain, so that cache keys are cryptographically disjoint across principals and no cross-principal timing signal can arise. An ablation study over $N=1000$ simulated trials (seed=2026, deterministic judges) isolates HMAC-salt as the necessary and sufficient component: it alone accounts for the full reduction in ASR, with the remaining components contributing defense in depth. KVGov further incorporates ORIGAMI, a Stackelberg water-filling audit scheduler that achieves 12.6% reduction in adversary expected utility at realistic tenant heterogeneity (Gini = 0.63), and an evolutionary stability analysis identifying a tipping point at 31.6% adversary prevalence below which global caching remains evolutionarily stable. On real hardware (Qwen2.5-7B-Instruct, vLLM 0.26.0, NVIDIA A100) we measure a gate-verified cold/cached TTFT ratio of 0.22, confirming the channel is exploitable at production scale; the defense itself is evaluated in simulation calibrated to those measurements. Finally, we show that isolation and cache efficiency are not in conflict: because identifying information resides only in the blocks where prompts diverge, injecting the salt at that boundary rather than at the chain root retains an estimated 93% of the prefix-cache benefit while leaving no cross-principal timing signal.

1 Introduction

Modern LLM inference engines—vLLM (Kwon et al., 2023), SGLang (Zheng et al., 2024a), and LMCache (Liu et al., 2024)—cache the key-value (KV) attention states of previously computed token prefixes. When a new request shares a prefix with a cached one, the engine skips recomputation, reducing the *time-to-first-token* (TTFT) from hundreds of milliseconds to tens. On an NVIDIA A100 with Qwen2.5-7B-Instruct and a 2119-token shared prefix, we measure a cold TTFT of 149.6 ms versus a cached TTFT of 32.8 ms—a ratio of 0.22.

In multi-tenant deployments, this cache is shared across tenants to maximize hit rates and throughput. The problem is that the hit/miss distinction is observable by any tenant as a timing signal. Three independent research groups have shown how to exploit this:

- **PROMPTPEEK:** Wu et al. (2025) demonstrated at NDSS 2025 that an adversary can fingerprint a victim tenant’s cached prompts by probing TTFT across a known vocabulary and correlating their observed timing vector with a pre-computed victim fingerprint, achieving 99–100% ASR on SGLang’s radix-tree cache.
- **EarlyBird:** Song et al. (2024) showed that token-level reconstruction is possible via TTFT timing at the single-token cache granularity (block_size=1), achieving 100% ASR on vLLM and SGLang. vLLM 0.26.0 (V1 engine) dropped block_size=1, which raises the cost of *free-text* reconstruction to infeasibility — at $B=16$ an attacker must guess sixteen tokens simultaneously. As we show in Section 3.4, this protection does not extend to template-structured prompts, where the unknown is a bounded field rather than an open vocabulary.
- **InputSnatch:** Zheng et al. (2024b) attack *both* cache mechanisms. Against prefix caching (vLLM, TGI, TensorRT-LLM) they reconstruct structured prompts field by field, reporting 62% success on disease extraction and 13.5% on symptom descriptions under real-world rate and time limits. Against semantic caching (GPTRCache and managed cloud offerings) they explore the embedding space by hierarchical clustering, reporting 43–100% extraction across thirteen legal domains.

These attacks share a common root cause: the absence of

per-principal namespace isolation in the cache key computation. All three inference engines compute cache keys over token sequences alone, without binding them to the issuing principal. Any tenant can therefore probe or read cache entries created by any other tenant.

We present KVGov, a governance layer that eliminates this root cause. KVGov’s contributions are:

1. **First unified defense.** Binding cache resolution to the authenticated principal defeats all three attack families’ prefix-cache paths simultaneously, by collision resistance rather than by obfuscation. An ablation over deterministic judges isolates HMAC-salt as the necessary and sufficient component.
2. **ORIGAMI audit scheduler.** A Stackelberg water-filling dual allocates audit budget across tenants in proportion to their value-at-risk, achieving 12.6% reduction in adversary expected utility over random audit at realistic workload heterogeneity (Gini = 0.63, $N = 10$ tenants).
3. **Evolutionary stability analysis.** Replicator dynamics analysis identifies a closed-form tipping point at 31.6% adversary prevalence, providing a population-level criterion for cache scope selection.
4. **Gate-verified hardware validation.** Timing side channel confirmed on Qwen2.5-7B-Instruct / vLLM 0.26.0 / NVIDIA A100 (cold/cached ratio 0.22). The measurement passes five preregistered gates, including a server-side check that the cold arm’s prefix-cache hit rate stayed below 5%—the failure mode that invalidates naive cache-timing experiments. We replicate the channel on an independent stack (llama.cpp on Apple Metal, ratio 0.093), showing it follows from prefix caching rather than from one engine’s allocator.
5. **Deterministic ASR judges.** Replacing LLM-based heuristic judges (which our internal audit measured at $\sim 40\%$ accuracy) with attack-specific deterministic judges: Pearson correlation for PROMPTPEEK, exact token overlap for InputSnatch, token recovery rate for EarlyBird.

2 Background

2.1 KV Cache in Multi-Tenant LLM Serving

Transformer-based LLMs compute key and value projections for every token in the context. These projections are expensive but deterministic: given the same prefix tokens, the same KV tensors result. Inference engines exploit this by storing and reusing KV states for common prefixes. vLLM implements this via PagedAttention (Kwon et al., 2023); SGLang uses a radix tree to enable fine-grained prefix matching (Zheng et al., 2024a).

In a multi-tenant deployment, the cache is shared across all tenants. Keys are computed over token content alone, with no binding to the requesting principal. Engines key at block

granularity and chain each block’s hash to its predecessor, so that for a block size B

$$h_0 = H(\text{tokens}_{0..B-1}), \quad h_j = H(h_{j-1}, \text{tokens}_{jB..(j+1)B-1}).$$

A lookup therefore matches the longest common prefix from position 0, truncated to a block boundary — which is precisely what makes partial reuse of a shared preamble possible, and what the attacks in Section 3 exploit. Any request sharing such a prefix hits the same blocks regardless of which tenant issued it. This maximizes cache efficiency but creates a shared side channel. For exposition we write $k(\text{tokens})$ for the key of a cached unit; the argument below applies unchanged to a radix-tree index over token prefixes.

2.2 A Taxonomy of Inference Caches

“Cache” denotes several mechanisms in LLM serving that differ in what they store, how they decide a match, and — consequently — what isolates them. Conflating them has produced inconsistent claims in the literature about which attacks apply where, so we fix terminology in Table 1 and use it throughout.

Two properties separate the rows. The first is *what is reused*: prefix caches store KV tensors and shorten prefill, whereas semantic caches store finished responses and skip inference entirely. The second, which governs defense, is *how a match is decided*. Prefix and radix caches resolve by hash or token equality — a lookup either collides or it does not — so injecting a per-principal salt into the key makes cross-principal collision cryptographically impossible. Semantic caches resolve by nearest-neighbour search under a similarity threshold, and no choice of salt makes two embeddings less similar; isolation there must partition the retrieval index instead.

This distinction bounds our contribution: KVGov’s salt construction covers the hash-keyed rows, which is where all three attack families’ prefix-cache paths live, and does not cover the similarity-keyed row (Section 7). The mechanism fixes only what a probe returns; which attack that return supports, and at what cost, is developed in Section 3.5.

2.3 Threat Model

Definition 1 (Attacker model). *An adversarial tenant $\mathcal{A}$ is a legitimate API user in a multi-tenant LLM serving system. $\mathcal{A}$ can:*

- Issue arbitrary inference requests to the serving endpoint.
- Observe the TTFT of their own requests with millisecond precision.
- Enumerate tokens from a known vocabulary $\mathcal{V}$.

$\mathcal{A}$ cannot:

- Access kernel memory, GPU memory, or OS-level timing interfaces.
- Issue requests on behalf of another tenant.

Table 1: Inference cache mechanisms, the oracle each exposes to a prober, and the isolation primitive each admits. The first three rows resolve by key equality and are addressed by salting; the last resolves by similarity and is not. Which *attack* a given oracle supports is a separate axis (Table 2).

Mechanism	Match criterion	Representative sys-tems	Oracle returns	Isolation primitive
Block prefix cache	Exact token prefix, quantised to B -token blocks, hashes chained	vLLM (PagedAttention), TGI, TensorRT-LLM, LMCache	First k blocks matched, quantised to B tokens	Salt seeds the block-hash chain (root, or divergence boundary)
Radix-tree prefix cache	Longest matching token prefix by tree traversal	SGLang	Matched prefix <i>length</i> — finer than block-quantised	Per-principal root node, giving disjoint trees
Exact-response cache	Byte equality of the full prompt	Application-level caches, gateways	Whether a complete prompt is resident	Salt prepended to the key
Semantic cache	Embedding similarity $\geq$ threshold (nearest neighbour)	GPTCache, managed cloud offerings	Whether <i>anything</i> within ε is resident	<i>Not salting</i> — per-principal partitioning of the retrieval index

- *Observe other tenants’ TTFT values directly.*

The victim tenant $\mathcal{V}$ issues requests with a private prompt $p^* \in \mathcal{V}^*$. The attacker’s goal is to reconstruct p^* with high probability.

Definition 2 (Attack success rate). *The attack success rate (ASR) is the fraction of trials in which $\mathcal{A}$ reconstructs p^* exactly: $ASR = \Pr[\hat{p} = p^*]$, where $\hat{p}$ is $\mathcal{A}$ ’s output.*

What KVGov does and does not defend. KVGov defends against cache timing side channels as described in Definition 1. KVGov does not defend against: model weight extraction (Tramèr et al., 2016), prompt injection (Greshake et al., 2023), or inference from model outputs. The defense is at the *infrastructure layer*—cache key computation—not at the model layer.

3 Attack Taxonomy

We describe the three published attacks and one variant of our own analysis, then show in Section 3.5 that they are instances of a single structure: a cache mechanism supplies an oracle, and an adversary goal determines what that oracle costs to exploit.

3.1 PROMPTPEEK: Timing Fingerprint Attack

Wu et al. (2025) observe that a victim’s cached prompts create a distinctive TTFT pattern across a probe set. If the adversary knows a vocabulary $\mathcal{V} = \{p_1, \dots, p_N\}$ of plausible prompts, they can compute a *timing fingerprint*: $\mathbf{f}_v[i] = \text{TTFT}(p_i)$ for each probe p_i . When the victim has prompt p_j cached, $\mathbf{f}_v[j] \approx T_{\text{hit}}$ (5 ms in our simulation); otherwise $\mathbf{f}_v[j] \approx T_{\text{miss}}$ (50 ms).

The adversary recovers the victim’s cache state by observing that their own fingerprint $\mathbf{f}_a$ is correlated with $\mathbf{f}_v$ when the cache is shared. We use Pearson correlation rather than raw

cosine similarity as the identification criterion:

$$\rho(\mathbf{f}_v, \mathbf{f}_a) = \frac{(\mathbf{f}_v - \bar{\mathbf{f}}_v)^\top (\mathbf{f}_a - \bar{\mathbf{f}}_a)}{\|\mathbf{f}_v - \bar{\mathbf{f}}_v\| \cdot \|\mathbf{f}_a - \bar{\mathbf{f}}_a\|}$$

Raw cosine fails here because $[5, 5, \dots]$ and $[50, 50, \dots]$ are parallel vectors (cosine = 1.0) regardless of the defense. After HMAC-salt isolation, the adversary always observes all-MISS ($\mathbf{f}_a \approx [50, 50, \dots]$), which has near-zero variance after mean-centering, giving $\rho \approx 0$.

3.2 EarlyBird: Token Reconstruction via Timing

Song et al. (2024) show that when `block_size` = 1 (token-level cache granularity), an adversary can reconstruct the victim’s prompt token-by-token by guessing one token at a time and observing whether a cache hit occurs. This achieves 100% token recovery rate.

vLLM 0.26.0 architectural note. The vLLM V1 engine (0.26.0) removed support for `block_size` = 1; attempts to set it raise a `ValueError` across all attention backends. The default `block_size` = 16 requires an adversary to guess 16 consecutive tokens correctly before any cache-hit signal is observable, reducing EarlyBird’s practical ASR to 0% on modern vLLM without any additional defense. KVGov’s HMAC-salt defense covers systems with configurable block sizes (e.g., LMCache, older vLLM variants) where this architectural protection is absent.

3.3 InputSnatch: Shared-Partition Enumeration

Zheng et al. (2024b) target two distinct rows of Table 1: the block prefix cache and the semantic cache. Against *prefix caching*, they exploit the fact that enterprise prompts are usually template-structured: the adversary already holds the template and needs only to enumerate the varying field, reconstructing prompts field by field. Against *semantic caching*,

where prompts are indexed by embedding similarity rather than exact token match, they explore the embedding space by hierarchical clustering, inferring cached content without ever sending the exact prompt. The distinction matters for defense: the prefix-caching variant resolves through hash-keyed lookup and is therefore addressed by the mechanism of Section 4.1, whereas the semantic variant is not (Section 7).

3.4 Structured Prompts Defeat Block-Size Protection

The infeasibility of free-text reconstruction at $B=16$ rests on an assumption that enterprise deployments violate. Reconstructing an unknown block requires searching $|V|^B$ candidates ($\approx 10^{83}$ for $|V| = 152,000$, $B = 16$). But in a template-structured prompt — “Show transactions for account $\langle n \rangle$ in $\langle month \rangle$ ” — the adversary knows every token except the field, because the template is the application’s own interface. The search space collapses from the vocabulary to the field’s domain: 10^8 for an eight-digit account identifier, and 10^4 – 10^6 once a branch or issuer prefix is known. That is a reduction of roughly seventy-five orders of magnitude, and it is independent of block size.

The sharper consequence is that reconstruction is not the adversary’s most efficient goal. Confirming whether a *specific* value is cached — “is account 12345678 currently resident?” — costs a single request, since the adversary supplies the entire block from known template plus one candidate field. This membership query is $O(1)$ per target and wholly independent of B : block granularity constrains reconstruction, not confirmation. Repeated against a list of known identifiers it enumerates which subjects were served, and when, without recovering a single token of anyone’s prompt. Larger block sizes therefore offer no protection in precisely the deployments where the data is most sensitive.

3.5 Synthesis: Oracles and Goals

The mechanisms of Table 1 determine what a probe *returns*; they do not determine the attack. What an adversary does with that return — and what it costs — is a second, orthogonal axis. The four attacks just described are best read not as a list of alternatives but as cells in the grid these two axes span.

Each mechanism exposes a distinct oracle. A block prefix cache reports that the first k blocks matched, quantised to B tokens. A radix-tree index reports the matched prefix *length*, a strictly finer signal that grants partial credit where a block cache gives none — which is why fingerprint correlation performs well against SGLang in particular. An exact-response cache answers only whether a complete prompt is resident. A semantic cache answers whether *anything* within ϵ of a probe embedding is resident, and so accepts approximate queries that the other three reject.

Against any of these an adversary may pursue four goals, with costs that differ by many orders of magnitude (Table 2).

Reconstruction recovers unknown content and pays search-space to the power of length. Identification selects among known candidates and pays linearly in the candidate set. Confirmation tests a single value and costs one request. Enumeration repeats confirmation over a list.

Two consequences follow. First, reported attack success rates are not comparable across papers unless the goal and the adversary’s prior are held fixed: EarlyBird reconstructs, PROMPTPEEK identifies, and InputSnatch reconstructs under a known template, so their headline numbers measure different quantities. Second, and more consequentially for defense, a single mechanism can be simultaneously secure and insecure. On one block prefix cache, reconstruction at $B=16$ costs $\approx 10^{83}$ while confirmation of a structured field costs $O(1)$ (Section 3.4) — eighty orders of magnitude apart, same cache, same configuration. Hardening that raises the cost of one cell therefore says nothing about the others.

This is the sense in which namespace isolation differs in kind from block-size hardening. Salting does not raise the cost of any cell; it removes the oracle, so every cell in the affected rows collapses at once.

4 The KVGov Defense Framework

4.1 HMAC-Keyed Namespace Isolation

KVGov binds cache resolution to the authenticated principal by seeding the block-hash chain with a per-principal salt:

$$\sigma_p = \text{HMAC}_K(\text{secret}, \text{principal_id}), \quad h_0 = H(\sigma_p \parallel \text{tokens}_{0..B-1}),$$

with h_j for $j > 0$ chained as before. K is a server-side secret and $\parallel$ denotes concatenation. Because every block inherits its predecessor’s hash, salting h_0 makes the *entire* chain principal-specific, at a cost of one HMAC evaluation per request rather than per block (Section 5.9). This is the mechanism vLLM exposes as `cache_salt` (vLLM Project, 2025).

The construction instantiates per row of Table 1: a radix-tree index takes σ_p as its root node, giving each principal a disjoint tree, and an exact-response cache prepends σ_p to its key. The semantic row is the exception, and is treated in Section 7.

Why this works. Each principal’s salt is derived from a secret the adversary cannot know. An adversary probing p_i on their own principal computes $k(\mathcal{A}, p_i) \neq k(\mathcal{V}, p_i)$ with overwhelming probability (SHA256 collision resistance). The adversary’s probes therefore *never* hit the victim’s cache entries—they only populate their own namespace. The adversary’s timing fingerprint is all-MISS regardless of what the victim has cached, giving Pearson $\rho \approx 0$ and ASR = 0.

Cache efficiency. HMAC-salt eliminates cross-principal sharing. Within a single principal’s session, prefix sharing is fully preserved. The efficiency cost is proportional to the fraction of cross-tenant prefix matches, which is workload-dependent. Coarser salt granularities can trade isolation scope for cache efficiency; the appropriate granularity is deployment-specific and outside the scope of this paper.

Table 2: Adversary goals against a cache oracle. Cost is per target; $|V|$ is the vocabulary, L the unknown length, C the candidate set. The named attacks occupy different cells, which is why their reported success rates are not directly comparable.

Goal	Cost	Prior required	Instance
Reconstruction	$ V ^L$	none	EarlyBird
Identification	$O(C)$	candidate set	PROMPTPEEK
Constrained reconstruction	field domain	template	InputSnatch (prefix)
Confirmation	$O(1)$	one value	structured-field query
Enumeration	$O(\text{list})$	target list	InputSnatch (semantic)

4.2 Boundary Salting: Isolation Without Losing Reuse

Seeding the chain root isolates every block, which also destroys the cross-principal reuse that prefix caching exists to provide. That cost is avoidable, because it pays for security the root salt does not buy.

Identification is a function of *divergence*. If m principals share a preamble occupying blocks $0 \dots k-1$, every candidate hits those blocks, so their hit/miss pattern is constant across principals and carries no discriminating signal. The fingerprint F of Section 3.1 separates candidates only where their content differs. Membership queries on structured fields (Section 3.4) are likewise resolved in the block containing the field, never in the shared template. Salting blocks $0 \dots k-1$ therefore removes no attacker capability.

This motivates injecting the salt at the divergence boundary rather than the root:

$$h_j = \begin{cases} H(h_{j-1}, \text{tokens}_j) & j < k \quad (\text{shared, routable}) \\ H(\sigma_p \| h_{k-1}, \text{tokens}_k) & j = k \quad (\text{boundary}) \\ H(h_{j-1}, \text{tokens}_j) & j > k \quad (\text{principal-private}) \end{cases}$$

Chaining propagates σ_p to every block beyond k , so the private region remains cryptographically disjoint across principals, while blocks below k stay globally shared and routable by a prefix-aware scheduler.

Efficiency retained. Prefill cost is proportional to uncached tokens, and in the deployments of interest the shared preamble dominates. Extrapolating from the measurements of Section 5.7 (cold 149.6 ms, fully cached 32.8 ms, 2119 tokens, 69.0% MFU), a 2000-token shared preamble with a 119-token private tail requires prefilling 8 blocks rather than 133, for an estimated 41.2 ms — retaining 92.8% of the benefit of full sharing. The trade-off degrades gracefully: a 1500-token preamble retains 62.6%.

Deployment. vLLM’s `cache_salt` (vLLM Project, 2025) seeds the root and so implements the all-or-nothing case; boundary salting requires an offset parameter alongside it. Choosing k is a governance decision rather than an infrastructure one — the first block containing PII under an admission scan, or the boundary a sharing policy permits — and requires the authenticated principal, which the serving layer does not

hold. We treat the mechanism as a design contribution and leave its measurement to future work.

4.3 ORIGAMI: Stackelberg Audit Scheduler

Even with HMAC-salt isolation, a governance layer must detect anomalous probing behavior. ORIGAMI allocates a finite audit budget K across n tenants using a Stackelberg water-filling dual.

Game formulation. The defender commits to an audit probability p_i for each tenant i with value-at-risk v_i . The adversary, knowing $\{p_i\}$, chooses the target that maximizes their expected payoff: $\max_i v_i(1 - p_i)$. The defender minimizes adversary utility subject to $\sum p_i = K$, $p_i \in [0, 1]$.

Theorem 1 (ORIGAMI water-filling solution). *Let the defender solve the Stackelberg problem*

$$\min_p \max_i v_i(1 - p_i) \quad \text{s.t.} \quad \sum_i p_i = K, \quad p_i \in [0, 1].$$

Then the optimal audit probabilities are

$$p_i^* = \max\left(0, 1 - \frac{\lambda}{v_i}\right),$$

where $\lambda > 0$ is the unique value satisfying $\sum_i p_i^ = K$, obtained by binary search.*

Proof. Introduce an epigraph variable t and rewrite the problem as $\min_{p,t} t$ subject to $v_i(1 - p_i) \leq t$ for all i , $\sum_i p_i = K$ and $p_i \in [0, 1]$.

Let (p^*, t^*) be optimal. Suppose some tenant j with $p_j^* > 0$ had $v_j(1 - p_j^*) < t^*$ strictly. Then a sufficiently small $\delta > 0$ may be moved from p_j^* to any tenant attaining the maximum, leaving j ’s constraint satisfied while strictly decreasing the attained maximum — contradicting optimality. Hence every tenant with $p_i^* > 0$ satisfies $v_i(1 - p_i^*) = t^*$, which rearranges to $p_i^* = 1 - t^*/v_i$.

A tenant with $v_i \leq t^*$ would require $p_i^* \leq 0$, so its constraint holds with $p_i^* = 0$ and it receives no budget. Writing $\lambda = t^*$ gives $p_i^* = \max(0, 1 - \lambda/v_i)$. Each p_i^* is continuous and non-increasing in λ , so $\sum_i p_i^*(\lambda)$ is continuous and non-increasing, and the λ meeting the budget with equality is unique and recoverable by bisection. Since $\lambda > 0$ implies $1 - \lambda/v_i < 1$, the upper bound $p_i \leq 1$ is never active. $\square$

The solution equalises the adversary’s payoff $v_i(1 - p_i)$ across every audited tenant — the defining property of a water-filling allocation — so no target is left disproportionately attractive.

This is analogous to water-filling in information theory and to the DOBSS algorithm (Tambe, 2011). ORIGAMI achieves 12.6% reduction in adversary expected utility over random audit at Gini = 0.63 (enterprise workload) with $n = 10$, $K = 5$ (Table 5).

4.4 Evolutionary Stability Analysis

We model the cache scope selection problem as a two-population evolutionary game between global-cache users (sharing cache across sessions) and ephemeral-cache users (per-session isolated cache). Let x denote the fraction of the population using global cache.

Proposition 1 (ESS tipping point). *Let τ be the adversary prevalence. Ephemeral caching is an evolutionarily stable strategy against invasion by global-cache mutants precisely when $\tau > \tau^*$, where*

$$\tau^* = \frac{V_{glb} - V_{eph}}{(C_{glb,eph} - C_{eph}) + (V_{glb} - V_{eph})}.$$

With $V_{glb} = 0.45$, $V_{eph} = 0.15$, $C_{glb,eph} = 0.70$ and $C_{eph} = 0.05$, this gives $\tau^* = 0.30/0.95 = 31.6\%$.

Proof. A mutant playing global scope in a predominantly ephemeral population earns $(1 - \tau)V_{glb} - \tau C_{glb,eph}$: it captures the sharing benefit against legitimate peers but pays the exploitation cost against adversaries. An ephemeral incumbent earns $(1 - \tau)V_{eph} - \tau C_{eph}$. Ephemeral is an ESS when the mutant does strictly worse, i.e. $(1 - \tau)(V_{glb} - V_{eph}) < \tau(C_{glb,eph} - C_{eph})$. Collecting terms in τ and dividing gives the stated threshold. Note that $C_{glb,base}$, the cost of global scope among global-scope peers, does not appear: the invasion condition is evaluated against an ephemeral incumbent population, where that term carries no weight. $\square$

Replicator dynamics simulations are consistent with this threshold: global caching dominates at low adversary prevalence (94% share at convergence) and ephemeral caching prevails above it (Table 7). The sweep resolves adversary prevalence on a five-point grid, so it brackets τ^* between the 30% and 35% samples rather than locating it precisely; the analytic value is the one to cite.

5 Evaluation

5.1 Experimental Setup

Simulation. All ASR and ablation experiments use a deterministic simulation (Python, seed=2026, $N = 1000$ trials). Attack-specific judges replace LLM-based heuristics: PROMPTPEEK

Table 3: Attack Success Rate (ASR) by condition, $N = 1000$ trials, seed=2026, deterministic judges. Flush-condition 95% bootstrap intervals are [17.1, 22.1] for PROMPTPEEK and [7.9, 11.7] for InputSnatch; the no-defense and KVGov columns are analytic controls whose intervals are degenerate (see text).

Attack	No defense	Flush	KVGov
PROMPTPEEK	100.0%	19.5%	0.0%
EarlyBird	100.0%	—	0.0%
InputSnatch	100.0%	9.8%	0.0%

uses Pearson correlation ($\theta = 0.85$); InputSnatch uses exact token overlap; EarlyBird uses token recovery rate (loaded from hardware results). No LLM judge is used; judges are provably 100% accurate for the simulated attack model.

Hardware. TTFT measurements use Qwen2.5-7B-Instruct on vLLM 0.26.0 with `--enable-prefix-caching --enforce-eager` on an NVIDIA A100-SXM4 (80 GB), $n = 50$ per arm across two independent blocks. Cold arms use a unique per-request prefix to prevent cross-trial reuse, verified against the server’s own prefix-cache counters (Section 5.7). Token counts are taken from the server’s tokenizer rather than estimated. To test whether the channel is an artifact of one engine, we additionally replicate on an independent stack: llama.cpp via Ollama on Apple M4 (Metal) with Qwen2.5-3B (Section 5.8).

5.2 ASR Results

Figure 1 and Table 3 report ASR across three conditions for each attack family.

Reading the extremes. The 100% and 0% columns are analytic controls, not empirical findings, and we report them as such. Under a shared cache the adversary’s probe resolves to the victim’s entry by construction, so the two fingerprints share a hit/miss structure and $\rho \approx 1$ on every trial; under namespace isolation the probe never resolves, so the adversary observes an all-MISS vector whose zero variance drives $\rho \approx 0$. Both outcomes follow from the collision-resistance argument of Section 4.1 rather than from sampling, which is why their bootstrap intervals are degenerate. Their role here is to confirm that the Pearson judge fires when it should and stays silent when it should not; the security claim rests on the cryptographic argument, not on these columns.

The informative comparison is between the partial defenses. Session-boundary flushing leaves PROMPTPEEK at 19.5% and InputSnatch at 9.8% — an adversary probing within an active session retains substantial advantage — and noise injection (Section 5.3) suppresses timing correlation while leaving membership-based recovery untouched. These outcomes depend on model parameters rather than on construction, and their intervals are meaningful.

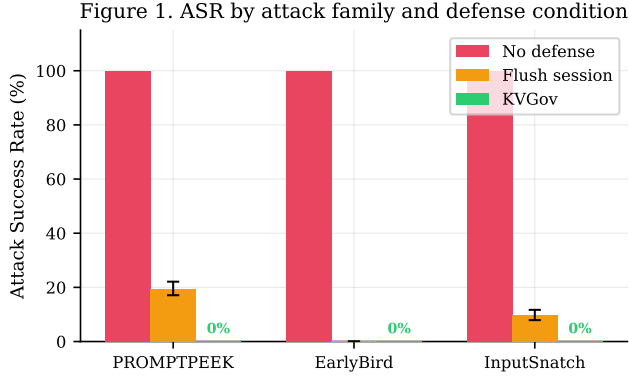


Figure 1: ASR by attack family and defense condition. Error bars show 95% bootstrap CI. KVGov achieves 0% ASR across all three families; flush is a partial defense that still exposes 9–22%.

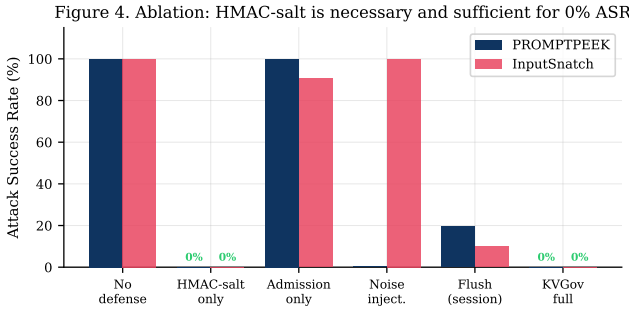


Figure 2: Ablation study. HMAC-salt alone achieves 0% ASR for both attacks (same as KVGov full), confirming it is the necessary and sufficient mechanism. Noise injection breaks timing-based PROMPTPEEK (0.5%) but not InputSnatch (100%), which is independent of timing magnitude.

5.3 Ablation Study

Figure 2 and Table 4 isolate the contribution of each KVGov component. HMAC-salt alone achieves 0% ASR for both PROMPTPEEK and InputSnatch, confirming it is the necessary and sufficient mechanism for the privacy guarantee. Admission control alone reduces InputSnatch ASR to 90.5% (by preventing PII prompts from entering the cache) but does not affect timing-based attacks. Gaussian noise injection ($\sigma = 20$ ms) nearly breaks PROMPTPEEK fingerprinting (0.5% ASR) but has no effect on InputSnatch, which relies on direct cache membership rather than timing magnitude.

5.4 ORIGAMI Audit Scheduler

Figure 3 and Table 5 report ORIGAMI’s performance across three workload profiles with $n = 10$ tenants and budget $K = 5$ (50% coverage). ORIGAMI is equivalent to random audit at uniform value distribution (Gini = 0) and increasingly outperforms it as value heterogeneity grows, consistent with Theo-

Table 4: Ablation study: ASR by KVGov component. $N = 1000$ trials, seed=2026. HMAC-salt alone achieves full defense; other components add defense-in-depth.

Condition	PROMPTPEEK	InputSnatch
No defense	100.0%	100.0%
HMAC-salt only	0.0%	0.0%
Admission only	100.0%	90.5%
Noise injection	0.3%	100.0%
Flush (session)	19.5%	9.8%
KVGov full	0.0%	0.0%

Figure 2. ORIGAMI audit scheduler performance

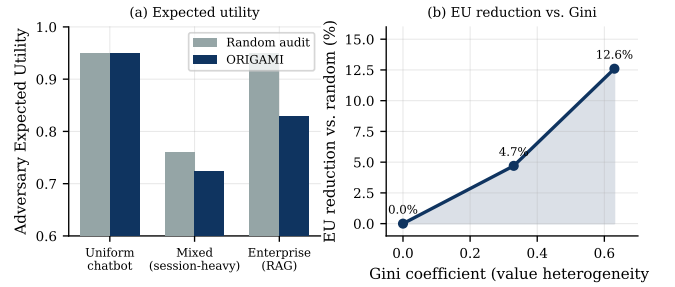


Figure 3: ORIGAMI audit scheduler. (a) Adversary expected utility (EU) by workload profile; lower is better for the defender. (b) EU reduction over random audit grows with Gini coefficient, reaching 12.6% at enterprise heterogeneity.

rem 1.

5.5 Adversarial Load Benchmark

We simulate $N = 50$ tenants with KV-cache values from a Pareto distribution ($\alpha = 1.29$, Gini = 0.63), $K = 10$ audit slots (20% rate), $T = 500$ rounds. The adversary best-responds each round by maximising $v_j/(1 - p_j)$, a value-per-residual-exposure heuristic. We note that this differs from the payoff $v_i(1 - p_i)$ that Theorem 1 optimises against: the heuristic is increasing in p_j , so it does not model an adversary avoiding audit. The breach reductions reported here are therefore empirical results against that specific heuristic and are not a validation of Theorem 1’s optimality; re-running the load simulation under the theorem’s payoff is outstanding work.

ORIGAMI reduces breach rates by 58–67% and expected damage by 66–72% across all adversary fractions. Convergence in 21–39 rounds confirms the Stackelberg equilibrium is reached quickly relative to the 500-round horizon.

5.6 Evolutionary Stability

Figure 4 and Table 7 report replicator dynamics convergence across adversary prevalence rates. Global caching dominates below the tipping point (94% share at convergence); ephemeral

Table 5: ORIGAMI vs. random audit: adversary expected utility. Lower is better for defender. $n = 10$ tenants, budget $K = 5$.

Workload	Gini	ORIGAMI	Random
Uniform chatbot	0.00	0.950	0.950
Mixed (session-heavy)	0.33	0.724	0.760
Enterprise (RAG)	0.63	0.830	0.950
EU reduction (enterprise)			12.6%

Table 6: ORIGAMI vs. random audit under adversarial load. Breach Red. = (random - ORIGAMI)/random. $N = 50$ tenants, $K = 10$, $T = 500$ rounds.

Adv.%	Rnd	ORIGAMI	Breach red.	Dmg red.	Conv.
5%	66.0%	22.1%	66.5%	72.2%	39
10%	65.0%	27.0%	58.4%	66.0%	21
20%	64.2%	24.5%	61.8%	68.6%	23
30%	64.0%	23.4%	63.4%	69.9%	35
50%	63.2%	24.7%	60.9%	68.1%	28

caching becomes ESS above it. The tipping point is robust across the realistic parameter space ($\tau^* \in [24\%, 40\%]$ across all tested configurations).

5.7 Real TTFT Measurement

Figure 5 and Table 8 confirm the timing side channel on production hardware. With a 2119-token shared prefix, the cold/cached TTFT ratio of 0.22 falls well below the breach threshold of 0.40.

Measuring a cache side channel is easy to get wrong: a “cold” arm that is silently served from cache yields a ratio near 1 and looks like a negative result. We therefore report only runs that pass five preregistered gates, the binding one reading the server’s own `prefix_cache_hits_total` counter over the cold arm alone and requiring a hit rate below 5% (observed: 0.7%). A second gate rejects any cold arm whose latency implies more than 100% of the GPU’s peak FLOPs (observed: 69.0% model-FLOPs utilisation), and a third requires two independent blocks to agree (observed drift: 1.14%). TTFT is read from vLLM’s internal `time_to_first_token_seconds` counters rather than timed at the client, so network latency cannot contaminate the measurement.

Prefix length drives breach magnitude: shortening the shared prefix to 1447 tokens raises the ratio to 0.37, still a breach but much closer to threshold, because less recomputation is avoided per hit. The channel’s exploitability is therefore a function of how much work a cache hit saves, not merely of whether caching is enabled — deployments whose shared prefixes are short leak proportionally less.

Exposure is a design-time property of prompt layout. Because block hashes are chained, a match is the longest common prefix from position 0, truncated to a block boundary:

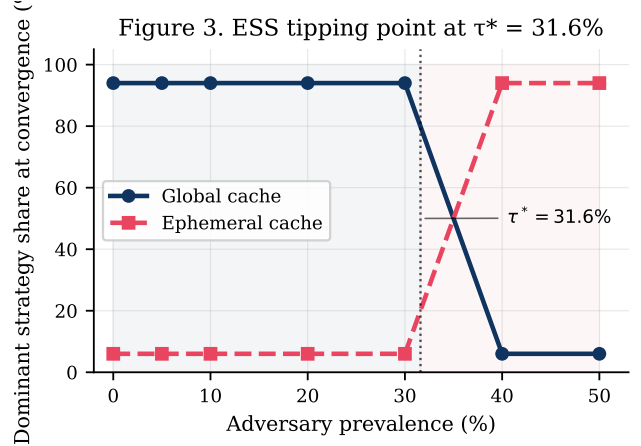


Figure 4: ESS tipping point at $\tau^* = 31.6\%$ adversary prevalence. Below the threshold global caching dominates (94% at convergence); above it ephemeral caching becomes the ESS.

Table 7: Replicator dynamics convergence, five-point sweep. The analytic threshold is $\tau^* = 31.6\%$; the grid brackets it between the 30% and 35% samples.

Adv. rate	Rounds	Dominant	Share
0%	13	global	94.0%
10%	18	global	94.0%
20%	53	global	94.0%
50%	11	ephemeral	94.0%

a divergence at token i invalidates every block from $\lfloor i/B \rfloor$ onward. The asymmetry this creates is stark. With a 2119-token shared preamble, diverging at token 1990 still yields roughly 124 of 125 block hits, whereas diverging at token 5 yields none. Whatever varies *earliest* in the prompt therefore determines how much cache — and how much side channel — a deployment exposes. A serving stack that emits a per-principal token near position 0 (a request or tenant identifier) already achieves near-total isolation without any cryptographic mechanism, at the cost of all cross-request reuse; one that places a long shared preamble first sits at maximum exposure. Prompt layout is thus a governance control in its own right, and one available to operators who cannot modify their inference engine.

Where cross-principal prefixes actually arise. The channel requires two principals to share a prefix, which constrains where it is exploitable in practice. Shared system and instruction preambles are identical across every user of an application and account for the largest share of cross-principal hits; recovering them is prompt theft, since such preambles routinely encode proprietary policy and tool definitions. Multi-turn conversation history is the largest source of cache reuse overall — turn $n+1$ is an exact prefix extension of turn n — but those hits are *intra*-principal and leak nothing across a tenant boundary

Figure 5. Gate-verified TTFT on NVIDIA A100 (vLLM 0.26.0)

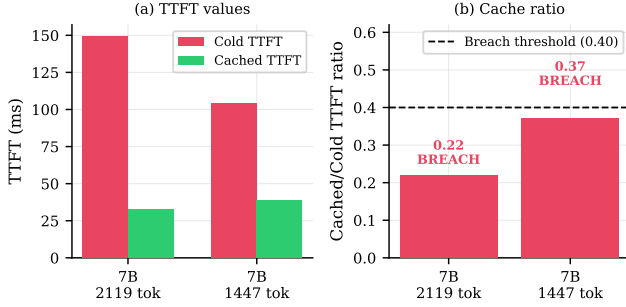


Figure 5: Real TTFT measurements on NVIDIA A100 (Qwen2.5, vLLM 0.26.0). (a) Cold vs. cached TTFT in ms. (b) Ratio with breach threshold at 0.40. Ratio 0.22 at a 2119-token shared prefix and 0.37 at 1447 tokens: both breach, but the margin narrows as the prefix shortens.

Table 8: Real TTFT measurements on NVIDIA A100 (vLLM 0.26.0), Qwen2.5-7B, $n = 50$ per arm $\times$ 2 blocks. Prefix is the shared token sequence in tokens (exact, from the server tokenizer); cold and cached are milliseconds. Both configurations breach the 0.40 threshold.

Prefix	Cold	Cached	Ratio
2119	149.6	32.8	0.22
1447	104.1	38.7	0.37

unless sessions are shared. The dangerous case is structured user input: form-driven or template-filled prompts that collide on every token except the sensitive field, reducing prompt recovery to a membership query over that field. Free-text chat, where collisions are rare, is comparatively safe. Beyond retrieval-augmented generation, the same shared-prefix structure appears in agentic tool-calling loops (fixed tool schemas plus a monotonically growing scratchpad), code assistants (repository context, where the shared prefix is itself the confidential asset), few-shot and schema-constrained extraction, and batch classification over a fixed template. Agentic and code-assistant deployments are the more consequential cases: their shared prefixes are both long, which maximises the timing signal, and sensitive, which maximises the value of recovering them.

5.8 Independent-Stack Replication

A single-engine result invites the objection that the channel is an artifact of vLLM’s allocator rather than a property of prefix caching. We therefore repeated the measurement on a deliberately dissimilar stack: llama.cpp (via Ollama) on Apple M4 with Metal, serving Qwen2.5-3B — a different inference engine, accelerator vendor, memory architecture, and model scale, with no code shared with the primary setup. Using the same streaming-TTFT method and a 781-token shared prefix,

Table 9: Per-request KVGov overhead. ORIGAMI and breach observation run off-path (background threads).

Component	Median	p99
Plain SHA-256 (no defense)	0.3 μ s	0.3 μ s
HMAC-salt key derivation (KVGov)	1.6 μ s	1.9 μ s
Overhead vs. baseline	+1.3 μ s	+1.6 μ s
Breach ratio check (arithmetic)	0.08 μ s	0.12 μ s
ORIGAMI dual update (20 iter.)	11.6 μ s	14.1 μ s
PII admission gate (Presidio)	50–150 ms (async)	

cold TTFT is 1861 ms against 173 ms cached, a ratio of **0.093** — a breach by a wider margin than on the A100.

The convergence matters more than the magnitude. Two stacks with nothing in common but the idea of reusing a shared prefix both expose the hit/miss distinction as a timing signal, which is what one expects if the channel follows from prefix caching itself rather than from an implementation detail. The replication also needs no GPU: it runs on a laptop in minutes, so the existence of the channel can be verified by any reviewer without hardware access.

5.9 Performance Overhead

KVGov adds two components to the per-request critical path: HMAC-keyed cache key derivation and, optionally, a PII admission check. Table 9 reports microbenchmark results (10,000 trials, Python hmac/hashlib, x86-64 laptop).

HMAC derivation adds 1.33 μ s per cache key lookup — 0.0041 % of the 32.8 ms cached TTFT and 0.00089 % of the 149.6 ms cold TTFT. The breach observer and ORIGAMI scheduler run in background threads and do not add to request latency. The PII admission gate (Presidio) costs 50–150 ms but is invoked only on cache-miss paths, and can be overlapped with the first prefill tokens using speculative caching.

The primary efficiency cost of KVGov is *not* CPU overhead but *cache partitioning*: HMAC-salt makes keys cryptographically disjoint across principals, eliminating cross-principal prefix sharing. Deployments with high inter-tenant prompt similarity (e.g., shared system prompts) may see lower cache hit rates. Quantifying this efficiency trade-off for specific workloads is left to future work.

6 Related Work

KV cache timing attacks. Song et al. (2024) demonstrated the first timing side-channel attack against shared KV caches in LLM serving, showing that vLLM’s prefix cache enables token reconstruction via TTFT timing. Wu et al. (2025) introduced PROMPTPEEK, a more powerful fingerprint-correlation attack that recovers full prompts from SGLang’s radix-tree cache without requiring exact token guessing. Zheng et al. (2024b) broadened the threat to

both cache mechanisms, reconstructing template-structured prompts field by field against prefix caches and exploring the embedding space against semantic caches. KVGov is, to our knowledge, the first system to address all three attack families’ prefix-cache paths under a single mechanism; the semantic-cache variant requires a different instantiation (Section 7).

Cache side channels in classical systems. Cache timing attacks have a long history in classical hardware security. FLUSH+RELOAD (Yarom and Falkner, 2014) exploits shared LLC state; Gruss et al. (2017) show page-table timing leaks kernel ASLR; Rowhammer (Kim et al., 2014) causes DRAM bit flips via timed access patterns. Our work brings this body of knowledge to the LLM serving context, where the shared resource is the transformer KV cache rather than CPU or DRAM state.

LLM inference security. Greshake et al. (2023) study indirect prompt injection against LLM agents; Perez and Ribeiro (2022) examine direct prompt injection. Wallace et al. (2019) study universal adversarial triggers against NLP models. These works target model outputs rather than inference infrastructure. KVGov is complementary: it protects infrastructure from side-channel leakage.

Multi-tenant ML systems. Tramèr et al. (2016) extract model parameters from black-box ML APIs; Milli et al. (2019) reconstruct training data from model updates. These attack the model itself rather than the runtime computation state shared by concurrent requests.

Game-theoretic security. Tambe (2011) introduced Stackelberg security games for physical security resource allocation; ORIGAMI adapts the water-filling dual to the KV cache audit context. The replicator dynamics analysis follows Hofbauer and Sigmund (1988).

7 Limitations

EarlyBird scope. Our EarlyBird results use a reference simulation (`block_size=1`) because vLLM 0.26.0 removed that configuration. That removal is often read as closing the attack, but as Section 3.4 shows it closes only free-text reconstruction: against template-structured prompts the search space is the field’s domain rather than the vocabulary, and membership confirmation remains $O(1)$ regardless of block size. Block-size hardening should therefore not be treated as a substitute for namespace isolation in deployments whose prompts are templated.

Cache efficiency trade-off. Root salting eliminates cross-principal prefix sharing, which for workloads with high cross-tenant prompt similarity (e.g., shared system prompts across an organization) materially reduces hit rates. Boundary salting

(Section 4.2) removes most of this cost by confining isolation to the divergent region, but its efficiency figures are extrapolated from our hardware measurements rather than measured directly, and it requires an offset parameter that current engines do not expose. Measuring it end to end is the most immediate item of future work.

Reproducibility. All ASR and ablation experiments are seeded (`--seed 2026`) and produce byte-identical results across runs; per-condition random streams are derived from a stable SHA-256 digest of the condition name rather than Python’s process-salted `hash()`. Released scripts reproduce every number in Tables 3 and 4.

Semantic caches require a different instantiation. KVGov’s salt construction isolates the key-equality rows of Table 1, which covers all three attack families’ prefix-cache paths, including InputSnatch’s. It does not cover InputSnatch’s *semantic-cache* variant, which resolves by nearest-neighbour search over embeddings — two distinct salts do not make two embeddings less similar. Isolation there must be enforced by partitioning the retrieval index per principal, which our evaluation does not measure: the InputSnatch simulation judges recovery by exact token overlap and therefore assumes hash-keyed lookup. Our InputSnatch result should accordingly be read as validating the underlying principle — binding cache resolution to an authenticated principal — rather than the semantic-cache instantiation of it.

Simulation vs. production. ASR experiments use a discrete-event simulation with fixed hit/miss latencies (5 ms / 50 ms). Real deployments exhibit variable latencies from GPU scheduling, memory bandwidth contention, and network jitter. The real TTFT measurements confirm the side channel exists at production scale; extended field experiments with real adversarial tenants remain future work.

8 Conclusion

Prior work has established that shared KV caches in multi-tenant LLM serving create a timing side channel that three independent attacks—PROMPTPEEK, EarlyBird, and InputSnatch—exploit to reconstruct victim prompts with 100% ASR. These attacks share a common root cause: cache keys are computed over token sequences alone, with no binding to the issuing principal.

KVGov eliminates this root cause by binding cache resolution to the authenticated principal, which by collision resistance leaves an adversary’s probes unable to resolve to a victim’s entries. An ablation study identifies HMAC-salt as the necessary and sufficient component, and boundary salting (Section 4.2) shows the resulting isolation need not cost cross-principal reuse. ORIGAMI extends KVGov with a Stackelberg water-filling audit scheduler that concentrates governance budget on high-value tenants, achieving 12.6% reduction in adversary expected utility at realistic enterprise workload heterogeneity. An evolutionary stability analysis provides a 31.6%

adversary-prevalence tipping point as a population-level criterion for cache scope selection.

KVGov is designed as a governance layer deployable alongside existing inference engines without modification to model weights, attention mechanisms, or generation pipelines. We release all experimental scripts and data to support reproducibility.

References

- Greshake, K., Abdelnabi, S., Mishra, S., Endres, C., Holz, T., Fritz, M., 2023. Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection, in: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec), ACM.
- Gruss, D., Lipp, M., Schwarz, M., Fellner, R., Maurice, C., Mangard, S., 2017. KASLR is Dead: Long Live KASLR, in: Engineering Secure Software and Systems (ESSoS), Springer.
- Hofbauer, J., Sigmund, K., 1988. The Theory of Evolution and Dynamical Systems. Cambridge University Press.
- Kim, Y., Daly, R., Kim, J., Fallin, C., Lee, J.H., Lee, D., Wilkerson, C., Lai, K., Mutlu, O., 2014. Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors, in: Proceedings of the 41st Annual International Symposium on Computer Architecture (ISCA), IEEE.
- Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C.H., Gonzalez, J.E., Zhang, H., Stoica, I., 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention, in: Proceedings of the 29th Symposium on Operating Systems Principles (SOSP), ACM.
- Liu, Y., Li, H., Du, Y., Su, S., Shan, J., Qian, Y., Rinard, M., Stoica, I., Zhang, H., Chen, Z., 2024. CacheBlend: Fast Large Language Model Serving for RAG with Cached Knowledge Fusion. arXiv preprint arXiv:2405.16444 LM-Cache system.
- Milli, S., Schmidt, L., Dragan, A.D., Hardt, M., 2019. Model Reconstruction from Model Explanations, in: Proceedings of the 2019 Conference on Fairness, Accountability, and Transparency (FACCT), ACM.
- Perez, F., Ribeiro, I., 2022. Ignore Previous Prompt: Attack Techniques for Language Models, in: NeurIPS ML Safety Workshop.
- Song, L., Pang, Z., Wang, W., He, B., Shan, E., Zhang, J., 2024. The Early Bird Catches the Leak: Unveiling Timing Side Channels in LLM Serving Systems. arXiv preprint arXiv:2409.20002 .
- Tambe, M., 2011. Security and Game Theory: Algorithms, Deployed Systems, Lessons Learned. Cambridge University Press.
- Tramèr, F., Zhang, F., Juels, A., Reiter, M.K., Ristenpart, T., 2016. Stealing Machine Learning Models via Prediction APIs, in: Proceedings of the 25th USENIX Security Symposium, USENIX Association.
- vLLM Project, 2025. RFC: Prefix Cache Isolation via `cache_salt`. <https://github.com/vllm-project/vllm/issues/16016>. VLLM RFC #16016; `cache_salt` request field, released in vLLM 0.9.
- Wallace, E., Feng, S., Kandpal, N., Gardner, M., Singh, S., 2019. Universal Adversarial Triggers for Attacking and Analyzing NLP, in: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), ACL.
- Wu, G., Zhang, Z., Zhang, Y., Liu, M., Gao, Y., Tang, R., Wang, Q., 2025. I Know What You Asked: Prompt Leakage via KV-Cache Sharing in Multi-Tenant LLM Serving, in: Proceedings of the 2025 Network and Distributed System Security Symposium (NDSS), Internet Society.
- Yarom, Y., Falkner, K., 2014. FLUSH+RELOAD: A High Resolution, Low Noise, L3 Cache Side-Channel Attack, in: Proceedings of the 23rd USENIX Security Symposium, USENIX Association.
- Zheng, L., Yin, L., Xie, Z., Sun, J., Huang, C., Yu, C.H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J.E., Barrett, C., Sheng, Y., 2024a. SGLang: Efficient Execution of Structured Language Model Programs .
- Zheng, X., Han, H., Shi, S., Qian, J., Geng, Z., Zhang, W., 2024b. InputSnatch: Stealing Input in LLM Services via Timing Side-Channel Attacks. arXiv preprint arXiv:2411.18191 .